

University Institute of Engineering (UIE)

Subject:	Computer Organization and Architecture
Case Study Title:	CPU Pipelining and Performance Improvement
Student Name:	SUBARNA LODH
UID/Roll Number:	25BAI10051
Class/Section	25BAI601

1. Introduction

The CPU is the bit that actually does the work in a computer system, it's the bit that executes the instructions and performs the operations. The way in which instructions are executed is a major factor in the performance of a processor.

One of the key techniques for improving CPU performance is instruction pipelining. The idea is to break down the instruction execution process into a number of stages so that multiple instructions can be processed at the same time. Rather than wait for one instruction to complete before starting the next, you can start processing the next instruction while the previous one is still in progress.

This is a case study of a processor executing 10 independent instructions through a 5 stage pipeline. It looks at the difference between sequential execution and ideal pipelined execution in terms of clock cycles and the resultant improvement in speed.

The study also looks at the problems encountered by real world pipelined processors i.e. structural, data and control hazards.

2. Objectives of the Case Study

The main objectives of this case study are:

1. To understand the concept and working of instruction pipelining.
2. To identify the five classic stages of CPU instruction execution.
3. To calculate the total clock cycles required for sequential execution.
4. To calculate the clock cycles required for ideal pipelined execution.
5. To determine the speedup achieved through pipelining.
6. To illustrate overlapping instruction execution using a pipeline timing chart.
7. To understand the hazards that affect the performance of real-world pipelines.

3. Problem Statement

A processor is required to execute 10 independent instructions. Each instruction passes through five stages of equal duration, with each stage requiring one clock cycle.

The processor is assumed to have an ideal five-stage pipeline without stalls, delays, or hazards.

The purpose of this case study is to compare the number of clock cycles required for sequential execution and ideal pipelined execution, calculate the performance improvement, and analyze the factors that may reduce pipeline efficiency in practical processors.

3.1 Given Data

- Number of instructions (n) = 10
- Number of pipeline stages (k) = 5
- Duration of the each stage = 1 clock cycle
- Pipeline type = Ideal five-stage pipeline
- Pipeline stalls = 0
- Instructions = Independent

4. Theoretical Background

4.1 What is Instruction Pipelining?

Instruction pipelining is a technique used in modern processors to improve CPU performance by overlapping the execution of multiple instructions.
In a non-pipelined processor, each instruction must complete all five stages before the next instruction can begin. This increases the total execution time.

In a pipelined processor, instruction execution is divided into multiple stages. While one instruction moves to the next stage, another instruction can enter the previous stage. This allows multiple instructions to be processed simultaneously.

4.2 Five Stages of Instruction Pipelining

A classic CPU pipeline consists of five stages:

1. Instruction Fetch (IF)
The processor fetche the instruction from memory using the Program Counter (PC).

2. Instruction Decode (ID)
The fetched instruction is decoded to understand which operation is going to be performed.

3. Execute (EX)
The Arithmetic Logic Unit (ALU) performs the arithmetic or logical operations, and calculates an address when required.

4. Memory Access (MEM)
The processor accesses data memory if the instruction requires a memory read or write operation.

5. Write Back (WB)
The result of the instruction is then written back to the destination register, if applicable.

4.3 Sequential Execution vs. Pipelined Execution

Sequential Execution:

In sequential execution, the processor completes all five stages of one instruction before starting another instruction.
For example:
Instruction 1: IF → ID → EX → MEM → WB
Instruction 2: IF → ID → EX → MEM → WB

This process continues until all instructions have been executed.

Pipelined Execution:

In pipelined execution, multiple instructions are processed in overlapping stages.

For example:

- Cycle 1: Instruction 1 performs IF.
- Cycle 2: Instruction 1 performs ID, while Instruction 2 performs IF.
- Cycle 3: Instruction 1 performs EX, Instruction 2 performs ID, and Instruction 3 performs IF.
- Cycle 4: Four instructions occupy four different pipeline stages.
- Cycle 5: Five instructions occupy all five pipeline stages.

After the pipeline is filled, ideally one instruction completes during every clock cycle.

4.4 Advantages of Instruction Pipelining

1. Increases instruction throughput.
2. Improves overall processor performance.
3. Allows better utilization of CPU resources.
4. Reduce the total execution time for multiple instructions.
5. Enables overlapping execution of instructions.

4.5 Limitations of Instruction Pipelining

Although pipelining improves processor performance, it also has certain limitations:

1. Pipeline hazards may cause delays or stalls.
2. Hardware design becomes more complex.
3. Branch instructions may interrupt the pipeline flow.
4. Data dependencies may delay subsequent instructions.
5. Unequal stage durations can reduce pipeline efficiency.

5. Case Study Questions and Answers

Q1. Explain Instruction Pipelining.

Answer: Instruction pipelining is a technique used in computer architecture to improve CPU performance by overlapping the execution of multiple instructions.
In this technique, instruction execution is divided into several stages, and each stage performs a specific operation. Instead of completing one instruction before starting the next, the processor allows multiple instructions to execute simultaneously in different stages.
For example, while the first instruction is being decoded, the second instruction can be fetched from memory. This overlapping process increases instruction throughput and reduces the total time required to execute multiple instructions.

Advantages of Instruction Pipelining:

1. Increases instruction throughput.
2. Improves CPU performance and efficiency.
3. Allows better utilization of processor resources.
4. Reduces the overall execution time for a sequence of instructions.

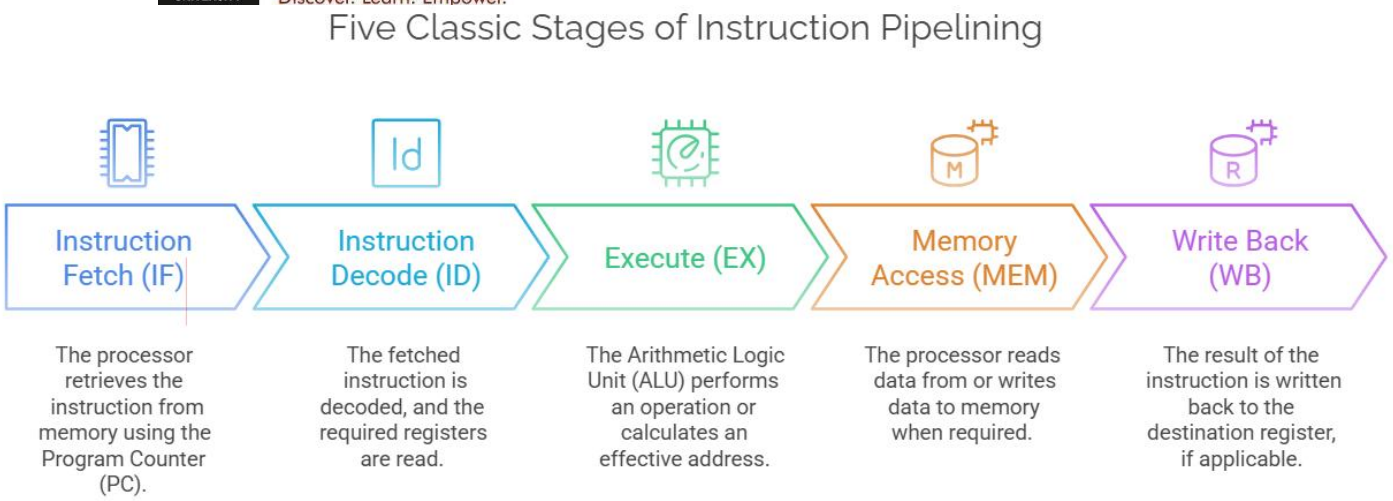
Thus, instruction pipelining improves processor performance by enabling overlapping instruction execution.

Q2. Name the Five Classic Pipeline Stages.

Answer: The five classic stages of instruction pipelining are:
1. Instruction Fetch (IF): The processor retrieves the instruction from memory using the Program Counter (PC).
2. Instruction Decode (ID): The fetched instruction is decoded to identify the operation to be performed, and the required registers are read.

3. Execute (EX): The Arithmetic Logic Unit (ALU) performs the required arithmetic or logical operation or calculates an effective address.

4. Memory Access (MEM): The processor reads data from or writes data to memory when required by the instruction.
5. Write Back (WB): The result of the instruction is written back to the destination register, if applicable.



Q3. How Many Cycles Are Needed Without Pipelining?

Answer: In a non-pipelined processor, each instruction must complete all five stages before the next instruction begins.

Given:

- Number of instructions (n) = 10
- Number of stages (k) = 5
- Each stage requires 1 clock cycle.

Formula:

Total Clock Cycle = Number of Instructions × Number of Stages = n × k

Calculation:

= 10 × 5

= **50 clock cycles**

Therefore, the processor requires **50 clock cycles** to execute all 10 instructions without pipelining.

Q4. How Many Cycles Are Needed in an Ideal Five-Stage Pipeline?

Answer: In an ideal pipelined processor, multiple instructions are executed in overlapping stages.

The first instruction requires five clock cycles to complete. After the pipeline is filled, one instruction completes in each subsequent clock cycle.

Given:

- Number of instructions (n) = 10
- Number of pipeline stages (k) = 5
- No stalls or hazards.

Formula:

Total Clock Cycles = k + (n − 1)

Calculation:

= 5 + (10 − 1)

= 5 + 9

= **14 clock cycles**

Therefore, the ideal five-stage pipeline requires only **14 clock cycles** to execute all 10 instructions.

This demonstrates a significant reduction in execution cycles compared to sequential execution.

Q5. What Are Structural, Data, and Control Hazards?

Answer: Pipeline hazards are conditions that prevent the next instruction from executing in its designated clock cycle. These hazards can introduce delays or stalls and reduce overall pipeline performance.

There are three major types of pipeline hazards:

1. **Structural Hazards:** It occurs when two or more than two instructions require the same hardware resouces at the same time.

Example: Two instructions attempting to access a single memory unit at the same time.
Solution: Using separate instruction and data memories or providing additional hardware resources.

2. Data Hazards: It occur when an instruction is dependend on the result of a previous instruction that has not yet completed execution.
Example:
Instruction 1: ADD R1, R2, R3
Instruction 2: SUB R4, R1, R5

Here, Instruction 2 requires the updated value of R1 produced by Instruction 1.
Solution: Data forwarding, pipeline stalling, or instruction scheduling.

3. Control Hazards: Control hazards occur when the processor encounters branch or jump instructions, making it uncertain which instruction should be executed next.
Example: A conditional branch instruction changes the normal sequence of instruction execution.
Solution: Branch prediction, early branch resolution, or pipeline flushing when necessary.

Conclusion:

Structural, data, and control hazards reduce pipeline efficiency by introducing stalls or additional execution cycles. Although the given case study assumes an ideal pipeline without hazards, real-world processors must use techniques such as forwarding, branch prediction, and resource management to minimize these delays.